AcmeCorp Collections and Reconciliation

Explanation for Sonic AI. A tool that chases overdue invoices by email, answers replies, and matches bank statements to invoices. A person approves every payment, and the LLM never decides or writes anything about money.

1. Architecture

React (Vite), plain CSS	nginx proxies /api	FastAPI backend	MongoDB
Pages: login, organization, customers, invoices, collections, reconciliation	Same origin in Docker, so no CORS in production	Routes and auth (main.py), all writes (store.py), matcher, evidence, agent, LLM client, mail	invoices, customers, payments (append-only), conversations, reminders, matches

Outside the backend: Gmail over SMTP (send) and IMAP (read, polled every 60 s or on a button), and an LLM that is either the Gemini free tier or a local *gemma4:e4b* through Ollama, chosen by one environment variable. The front end and back end are separate apps that talk only through the API. **Tech stack:** Python 3.12, FastAPI, Motor, rapidfuzz, httpx, itsdangerous; React 18 and Vite; MongoDB 7; Docker Compose; pytest.

2. Bank reconciliation (local algorithm, no LLM)

Each credit row goes through hard filters, then an additive score. Filters: same currency, invoice status *sent* with a balance, and invoice date not after the payment date. Draft invoices are never candidates.

Signal	Points	How it is found
Invoice number in narration	+50 (-25 if it names a different invoice)	Regex for INV-nnnn, INV nnnn, RE nnnn; only numbers that exist
Payer name	30 x similarity	0.5 x weighted token coverage + 0.25 x rapidfuzz token-set ratio + 0.25 x char 3-gram TF-IDF cosine
Amount	+30 exact; +28 TDS; +8 partial (+10 if narration says PART); -30 if over balance	Integer minor units. TDS = total minus 10% of subtotal, India and INR only, exact
Date	+5 within 30 days of due date, else +2	Payment date against due date
Ties	-10	Two equal invoices of one customer: oldest due date is suggested at lower confidence

Why these techniques. Narrations abbreviate (*ORBIS PKG*, *SAFFRON PEAK HOSP*). Token coverage with prefix and subsequence matching handles abbreviations, weighted by token rarity so a generic word such as *retail* counts less. N-gram TF-IDF tolerates typos and token-set ratio ignores word order. I rejected Levenshtein alone (fails on abbreviations) and sentence embeddings (heavy dependency for setup; I did not benchmark them, so that is a judgement). Weights were set by hand against the sample and my synthetic cases.

Precision first. A match is suggested at score 45 or more. An unknown payer whose amount equals exactly one open invoice is shown as *low confidence*, capped at 40, and the Approve button stays disabled until the person ticks a confirmation. Several invoices paid together are found by an exact subset-sum over one known payer's open invoices (2 to 4 invoices). On approval the payment is split across them. A TDS close is preselected only on an exact amount; any other shortfall defaults to *leave balance open*.

Customer emails as evidence (bonus). An email from a known customer links to a row when it contains the row's bank reference or names the suggested invoice. A reference quoted by the invoice's owner adds 15 points, or promotes a low-confidence row to a match at 75. A reference quoted by a different customer subtracts 15 and shows a warning. This is code, not the LLM. The email is shown beside the row.

Evaluation. *eval/run.py* scores 24 labelled rows (the 14 sample rows and 10 synthetic edge cases: typos, TDS, ties, wrong currency, payment before invoice date, a draft-only amount, a two-invoice payment) and writes *eval/report.md*. Full matcher: 100% precision, 100% recall, 0 wrong matches. Name-only baseline: 39% precision (11 wrong). Amount-only baseline: 54% precision (6 wrong). These are my own labels, so this is a sanity check and not a benchmark. One of my labels was wrong at first (an ambiguous short payment); I fixed the label, not the matcher.

3. Collections agent

Reminders are a fixed template sent by a button: one email per customer listing overdue invoices (status sent, balance above zero, due before today; *DEMO_TODAY* can pin the date). A 3-day cooldown per invoice stops repeats. **Inbound mail** follows this order:

- Automated mail (out-of-office, bounce, *Auto-Submitted* or bulk headers) is recorded and never answered.
- The sender must exactly match a customer's contact email. Unknown senders go to *Needs attention* and no customer data reaches the LLM.

- The LLM sees only that customer's non-draft invoices and returns JSON: an intent (balance question, claims paid, promises payment, dispute, extension request, other), a confidence, and a draft that uses placeholders such as *{INV-1004.balance}*.
- Code fills every figure from the database. A draft containing any digit of its own, an unknown placeholder, or another customer's invoice is discarded and routed to a person.
- Balance questions, payment claims and promises get an automatic reply. Claims are noted but nothing is marked paid until the money appears on a bank statement. Disputes, extension requests, low confidence and LLM failures go to *Needs attention*.

Emails are treated as untrusted data: instructions inside them cannot widen the context or change the rules (tested with a prompt-injection case).

4. Data model, correctness and security

Money is stored as integer paise or cents, never floats. Approving a match first claims the match row (so a double click returns 409), then updates each invoice with a conditional write that only succeeds if the balance is unchanged, and appends a payment record (amount, TDS withheld, narration). This avoids multi-document transactions, which need a replica set. Login is one admin account from environment variables with a signed, HTTP-only session cookie. Login, upload, reminder and inbox routes are rate limited. Inputs are validated, CSV uploads are capped at 1 MB, CORS is restricted, and secrets live only in *.env*.

5. Testing and honest limits

30 tests cover the matcher, the agent validator, the API (including double approval, TDS and the low-confidence guard), reminders and evidence linking. They use a fake LLM and an in-memory database. The live Gmail and Gemini/Ollama calls are not covered by automated tests; the README lists the manual steps. Known gaps: replies depend on the model (anything unverifiable goes to a person, so a weak model sends fewer replies, not wrong ones); TDS assumes India and 10% of the subtotal; statement dates must be YYYY-MM-DD; a short payment from a customer with several open invoices is left to a human; the cookie is not marked Secure on plain HTTP; one backend instance is assumed because the inbox poller runs in-process.